CS301: Computability and Complexity Theory (CC)

Lecture 5: Decidability

Dumitru Bogdan

Faculty of Computer Science
University of Bucharest

November 1, 2024

Table of contents

1. Previously on CS301

2. Context setup

3. Undecidability (cont)

Section 1

Previously on CS301

The relationship among classes of languages

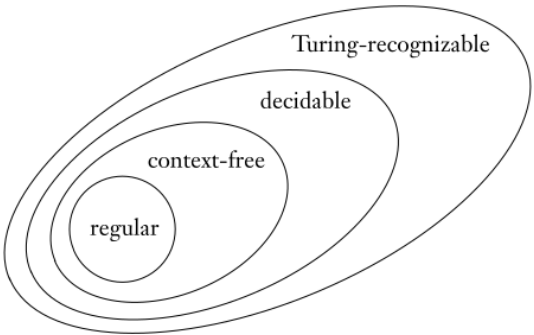


Figure: The relationship among classes of languages

The Diagonalizaton Method

Definition

A set A is **countable** if either it is finite or it has the same size as $\mathcal{N}$

Example

Let $\mathcal{Q} = \{\frac{m}{n} | m, n \in \mathcal{N}\}$ be the set of positive rational numbers. $\mathcal{Q}$ seems to be much larger than $\mathcal{N}$. Yet, these two sets have the same size according to our definition. We give a correspondence with $\mathcal{N}$ to show that $\mathcal{Q}$ is countable. One easy way to do so is to list all the elements of $\mathcal{Q}$. Then we pair the first element on the list with the number 1 from $\mathcal{N}$, the second element on the list with the number 2 from $\mathcal{N}$, and so on. We must ensure that every member of $\mathcal{Q}$ appears only once on the list.

To get this list, we make an infinite matrix containing all the positive rational numbers. The ith row contains all numbers with numerator i and the jth column has all numbers with denominator j. So the number $\frac{i}{j}$ occurs in the ith row and jth column.

Example

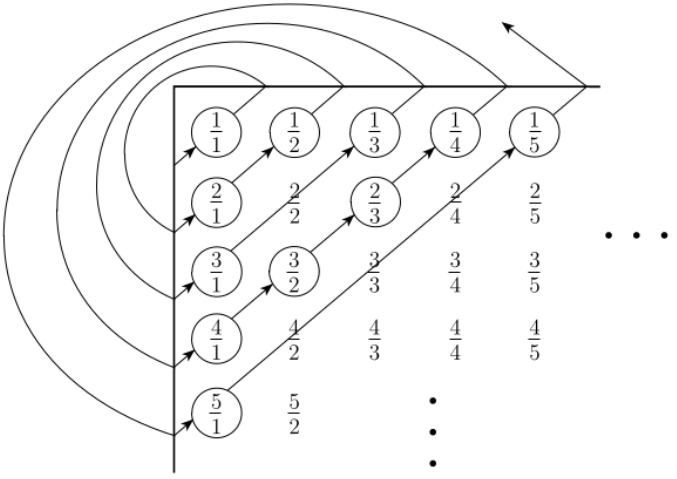


Figure: A correspondence of $\mathcal{N}$ and $\mathcal{Q}$

$\mathcal{R}$ is uncountable

Theorem

$\mathcal{R}$ is uncountable

Proof idea: Suppose that the correspondence f exists. Let $f(1) = 3.14159...$, $f(2) = 55.55555...$, $f(3) = \ldots$, and so on, just to make up some values for f. Then f pairs the number 1 with $3.14159\ldots$, the number 2 with $55.55555\ldots$, and so on. The following table shows a few values of a hypothetical correspondence f between $\mathcal{N}$ and $\mathcal{R}$.

n	$f(n)$
1	3.14159...
2	55.55555...
3	0.12345...
4	0.50000...
⋮	⋮

Section 2

Context setup

Context setup

Corresponding to Sipser 4.2 & 5.1

Context setup

- We continue to investigate the power of algorithms to solve problems
- We demonstrate certain problems that can be solved algorithmically and others that cannot
- Next, we examine several additional unsolvable problems
- We introduce the primary method for proving that problems are computationally unsolvable: **reducibility**

Section 3

Undecidability (cont)

Some languages are not Turing-recognizable

The preceding theorem has an important application to the theory of computation. It shows that some languages are not decidable or even Turing-recognizable, for the reason that there are uncountably many languages yet only countably many Turing machines. Because each Turing machine can recognize a single language and there are more languages than Turing machines, some languages are not recognized by any Turing machine. Such languages are not Turing-recognizable, as we state in the following theorem:

Theorem

Some languages are not Turing-recognizable

Some languages are not Turing-recognizable

Proof.

To show that the set of all TMs is countable, we first observe that the set of all strings Σ^* is countable for any alphabet Σ. With only finitely many strings of each length, we may form a list of Σ^* by writing down all strings of length 0, length 1, length 2, and so on.

The set of all TMs is countable because each TM M has an encoding into a string $< M >$. If we simply omit those strings that are not legal encodings of TMs, we can obtain a list of all TMs.

To show that the set of all languages is uncountable, we first observe that the set of all infinite binary sequences is uncountable. An **infinite binary sequence** is an unending sequence of 0s and 1s. Let $\mathcal{B}$ be the set of all infinite binary sequences. We can show that $\mathcal{B}$ is uncountable by using a proof by diagonalization similar to the one we used to show that $\mathcal{R}$ is uncountable.

Some languages are not Turing-recognizable

Proof.

Let $\mathcal{L}$ be the set of all languages over alphabet Σ. We show that $\mathcal{L}$ is uncountable by giving a correspondence with $\mathcal{B}$, thus showing that the two sets are the same size.

Let $\Sigma^* = \{s_1, s_2, s_2, \dots\}$. Each language $A \in \mathcal{L}$ has a unique sequence in $\mathcal{B}$. The ith bit of that sequence is a 1 if $s_i \in A$ and is a 0 if $s_i \notin A$, which is called the *characteristic sequence* of A. For example, if A were the language of all strings starting with a 0 over the alphabet $\{0, 1\}$, its characteristic sequence χ_A would be

$$\begin{array}{rccccccccccc} \Sigma^* = \{ & \varepsilon, & 0, & 1, & 00, & 01, & 10, & 11, & 000, & 001, & \cdots & \} \ ; \\ A = \{ & & 0, & & 00, & 01, & & & 000, & 001, & \cdots & \} \ ; \\ \chi_A = & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & \cdots & . \end{array}$$

Some languages are not Turing-recognizable

Proof.

The function $f : \mathcal{L} \to \mathcal{B}$, where $f(A)$ equals the characteristic sequence of A, is bijective. Therefore, as $\mathcal{B}$ is uncountable, $\mathcal{L}$ is uncountable as well. Thus we have shown that the set of all languages cannot be put into a correspondence with the set of all TMs. We conclude that some languages are not recognized by any TM. □

A_{TM} is undecidable

Now we have all we need to prove

Theorem

A_{TM} is undecidable

Proof.

We assume that A_{TM} is decidable and obtain a contradiction. Suppose that H is a decider for A_{TM}. On input $< M, w >$, where M is a TM and w is a string, H halts and accepts if M accepts w. Furthermore, H halts and rejects if M fails to accept w. In other words, we assume that H is a TM, where:

$$H(< M, w >) = \begin{cases} \textit{accept if } M \textit{ accepts } w \\ \textit{reject if } M \textit{ does not accept } w \end{cases}$$

A_{TM} is undecidable

Proof.

Now we construct a new TM D with H as a subroutine. This new TM calls H to determine what M does when the input to M is its own description $< M >$. Once D has determined this information, it does the opposite. That is, it rejects if M accepts and accepts if M does not accept. The following is a description of D

$D =$ On input $< M >$, where M is a TM:

1. Run H on input $< M, < M >>$
2. Output the opposite of what H outputs: if H accepts, reject; and if H rejects, accept

A_{TM} is undecidable

Proof.

Don't be confused by the notion of running a machine on its own description! That is similar to running a program with itself as input, something that does occasionally occur in practice. For example, a compiler is a program that translates other programs. A compiler for the language Python may itself be written in Python, so running that program on itself would make sense. To summarise we have

$$D(< M >) = \begin{cases} \textit{accept if } M \textit{ does not accepts } < M > \\ \textit{reject if } M \textit{ accepts } < M > \end{cases}$$

A_{TM} is undecidable

Proof.

What happens when we run D with its own description $< D >$ as input? In that case, we get

$$D(< D >) = \begin{cases} \textit{accept if } D \textit{ does not accepts } < D > \\ \textit{reject if } D \textit{ accepts } < D > \end{cases}$$

No matter what D does, it is forced to do the opposite, which is obviously a contradiction. Thus, neither TM D nor TM H can exist □

A_{TM} is undecidable

Where is the diagonalization in the proof? It becomes apparent when you examine tables of behavior for TMs H and D. In these tables we list all TMs down the rows: $M_1, M_2, \ldots$ and all their descriptions across the columns: $< M_1 >, < M_2 >, \ldots$ The entries tell whether the machine in a given row accepts the input in a given column. The entry is accept if the machine accepts the input but is blank if it rejects or loops on that input

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\cdots$
M_1	*accept*		*accept*		
M_2	*accept*	*accept*	*accept*	*accept*	
M_3					$\cdots$
M_4	*accept*	*accept*			
$\vdots$			$\vdots$		

A_{TM} is undecidable

In the following figure, the entries are the results of running H on inputs corresponding to previous. So if M_3 does not accept input $< M_2 >$, the entry for row M_3 and column $< M_2 >$ is reject because H rejects input $< M_3, < M_2 >>$

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\cdots$
M_1	*accept*	*reject*	*accept*	*reject*	
M_2	*accept*	*accept*	*accept*	*accept*	$\cdots$
M_3	*reject*	*reject*	*reject*	*reject*	
M_4	*accept*	*accept*	*reject*	*reject*	
$\vdots$			$\vdots$		

A_{TM} is undecidable

In the following figure, we add D. By our assumption, H is a TM and so is D. Therefore, it must occur on the list of $M_1, M_2, \ldots$. Note that D computes the opposite of the diagonal entries. The contradiction occurs at the point of the question mark where the entry must be the opposite of itself

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\cdots$	$\langle D \rangle$	$\cdots$
M_1	*accept*	*reject*	*accept*	*reject*		*accept*	
M_2	*accept*	*accept*	*accept*	*accept*	$\cdots$	*accept*	$\cdots$
M_3	*reject*	*reject*	*reject*	*reject*		*reject*	
M_4	*accept*	*accept*	*reject*	*reject*		*accept*	
$\vdots$			$\vdots$		$\ddots$		
D	*reject*	*reject*	*accept*	*accept*		?	
$\vdots$			$\vdots$				$\ddots$

Not Turing-recognizable

We showed that A_{TM} is Turing-recognizable. The following theorem shows that if both a language and its complement are Turing-recognizable, the language is decidable. Hence for any undecidable language, either it or its complement is not Turing-recognizable. Recall that the complement of a language is the language consisting of all strings that are not in the language. We say that a language is **co-Turing-recognizable** if it is the complement of a Turing-recognizable language.

Theorem

A language is decidable iff it is Turing-recognizable and co-Turing-recognizable

Rephrasing: a language is decidable exactly when both it and its complement are Turing-recognizable

Not Turing-recognizable

Proof.

We have two directions to prove. First, if A is decidable, we can easily see that both A and its complement A are Turing-recognizable. Any decidable language is Turing-recognizable, and the complement of a decidable language also is decidable. For the other direction, if both A and $\overline{A}$ are Turing-recognizable, we let M_1 be the recognizer for A and M_2 be the recognizer for $\overline{A}$. The following TM M is a decider for A

M = On input w:

1. Run both M_1 and M_2 in parallel
2. If M_1 accepts, *accept*; if M_2 accepts, *reject*

Not Turing-recognizable

Proof.

Now we show that M decides A. Every string w is either in A or $\overline{A}$. Therefore, either M_1 or M_2 must accept w. Because M halts whenever M_1 or M_2 accepts, M always halts and so it is a decider. Furthermore, it accepts all strings in A and rejects all strings not in A. So M is a decider for A, and thus A is decidable. □

Not Turing-recognizable

Theorem

$\overline{A_{TM}}$ is not Turing-recognizable.

Proof.

We know that A_{TM} is Turing-recognizable. If $\overline{A_{TM}}$ also were Turing-recognizable, A_{TM} would be decidable. We know that A_{TM} is not decidable, so $\overline{A_{TM}}$ must not be Turing-recognizable. □